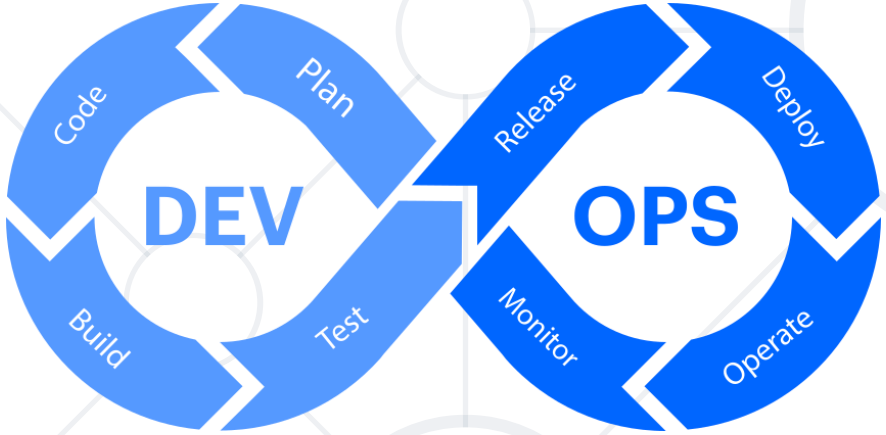
Build Pipelines

Build and Publish Artifacts



SoftUni Team

Technical Trainers

Software University

SoftUni

Software University

https://softuni.bg

Have a Question?

Software University

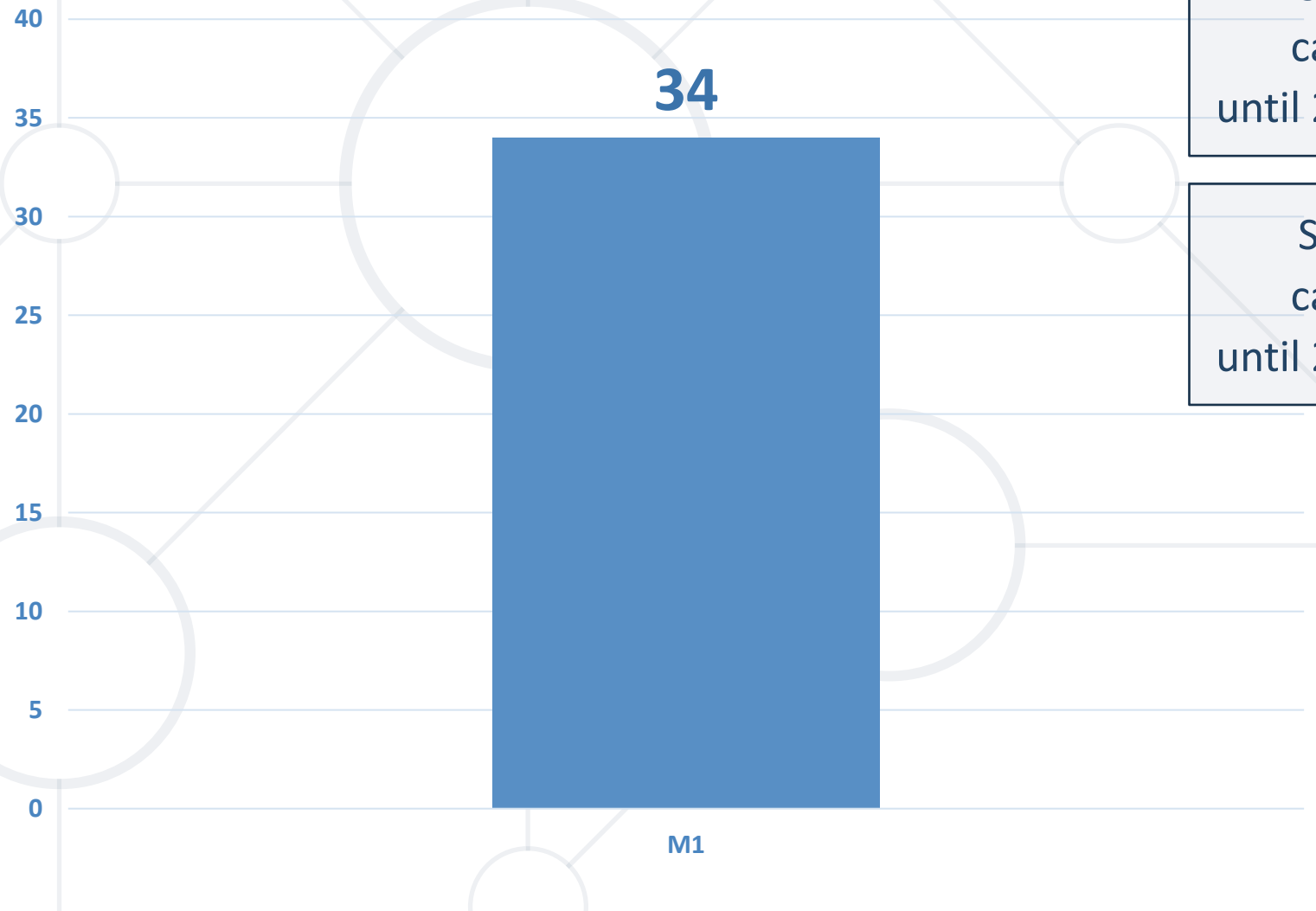
sli.do

#DevOps-CI-CD

facebook.com/groups/

cicdmonitoringjanuary2026

Homework Progress



Solutions for M1
can be submitted
until 23:59 on 28.01.2026

Solutions for M2
can be submitted
until 23:59 on 04.02.2026

Previous Module (M1)

Quick Overview

What We Covered

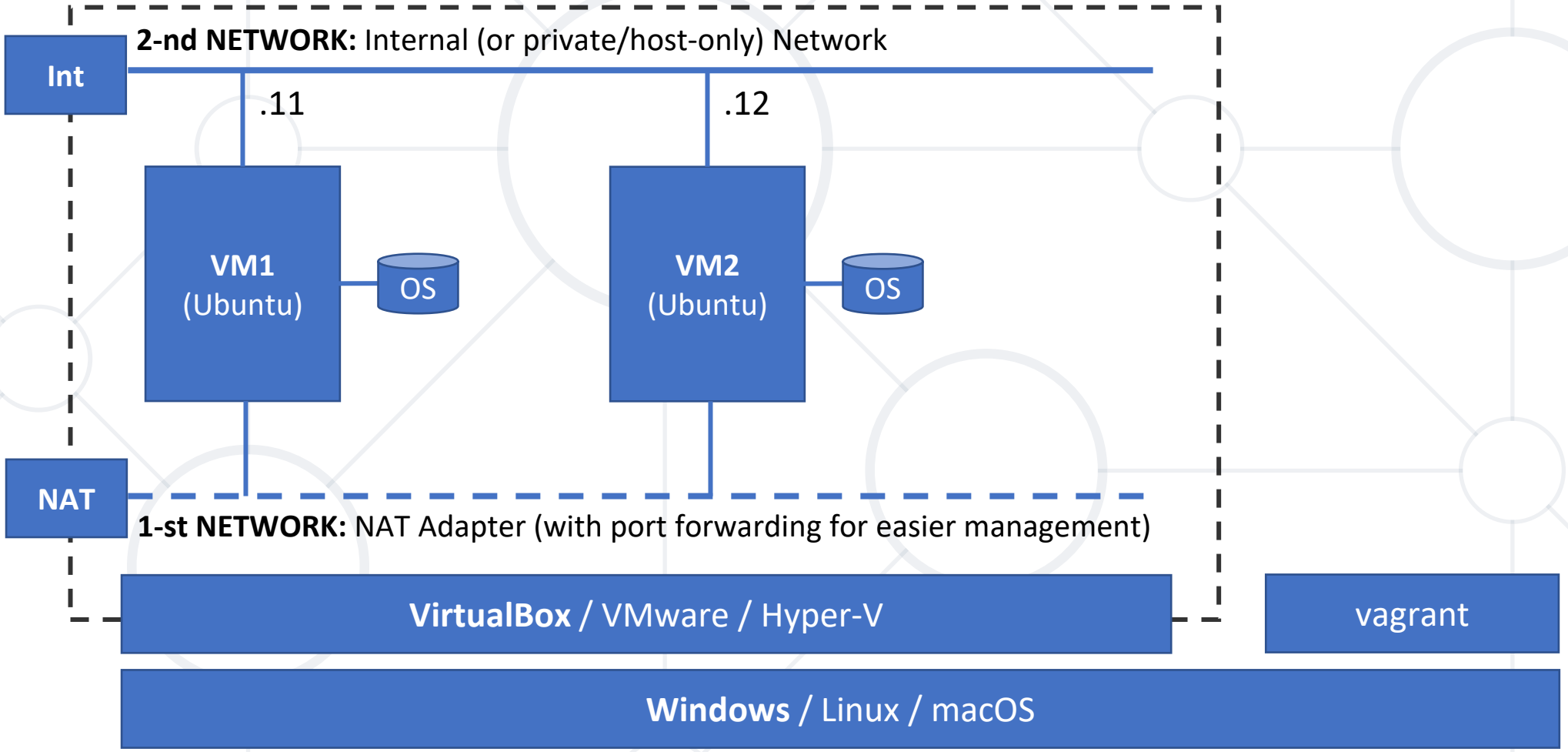
1. SDLC, DevOps, and CI/CD
2. Tooling Landscape
3. Our Roadmap
4. Start Simple

This Module (M2)

Topics and Infrastructure

Table of Contents

Lab Infrastructure



Version (Source) Control

Git. Files Lifecycle. Basic Commands

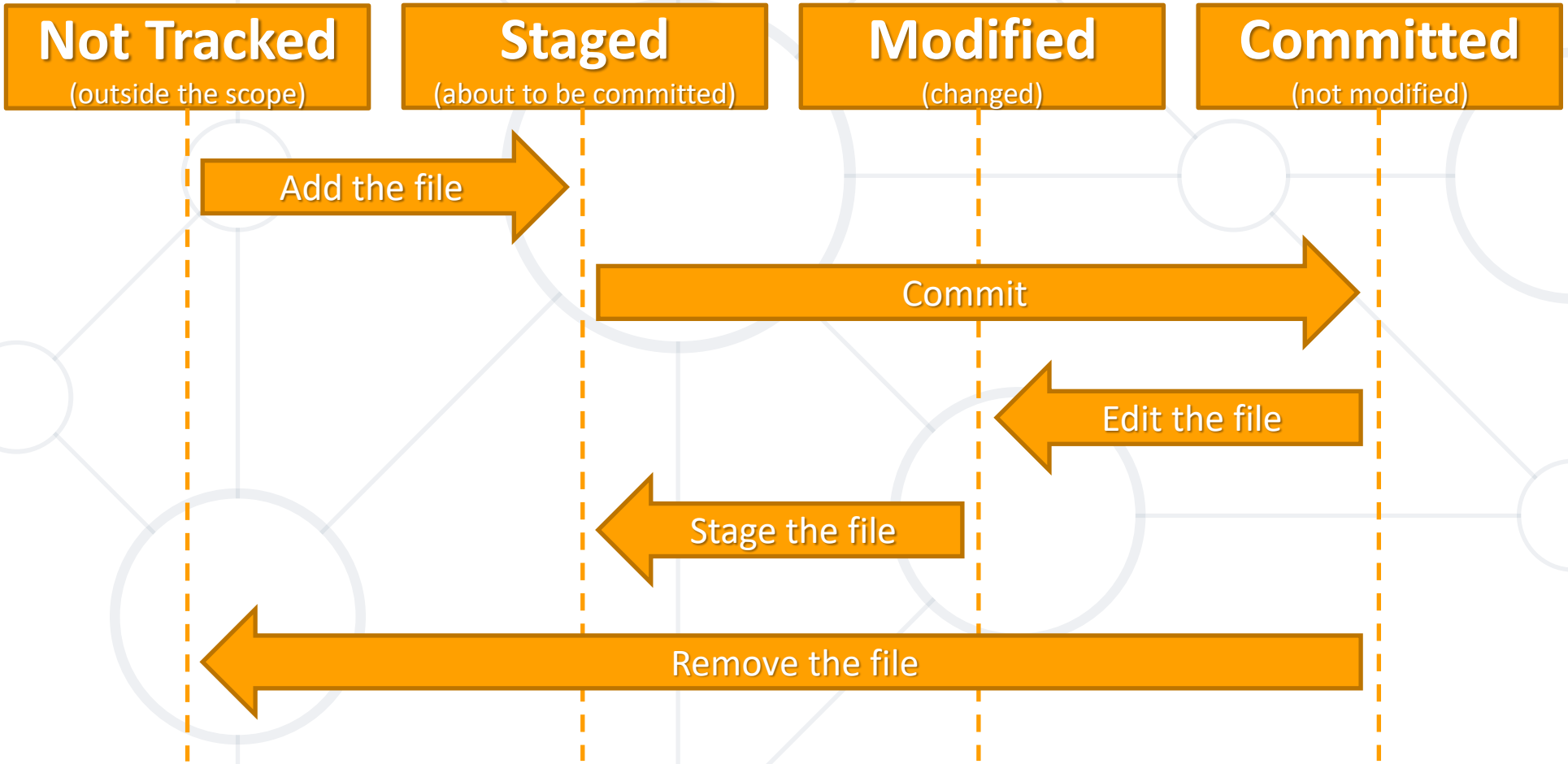
Version vs Source Control

- **Source (code) Control Systems (S(C)CS)**
  - Focus on managing access and changes to source code files
- **Version Control Systems (VCS)**
  - Management of changes in a software project over time
  - Besides source code control, it tracks other project elements
- We often use those interchangeably, and it is not an issue

Git: General Information

- **Distributed Version Control System** (**DVCS**)
  - A combination of local repositories and a shared one
- In contrast to **Centralized Version Control System** (**CVCS**)
  - Central repository that stores the version history of the project
- Created by the Linux development community in 2005
- Can be used on-premise and in the cloud
- Snapshot based
- Three states - **Committed**, **Modified**, and **Staged**

Git Files Lifecycle



Basic Git Commands

- Create an empty Git repository

```
git init
```

- Clone an existing repository

```
git clone https://github.com/user/repo
```

- Show repository objects

```
git show
```

- Show different states of files in working directory and staging

```
git status
```

Basic Git Commands

- Add files from working directory to staging

```
git add file.txt
```

- Remove a file from staging and working directory

```
git rm old_file.txt
```

- Move or rename file

```
git mv old_file.txt new_file.txt
```

- Commit the staged changes

```
git commit
```

Basic Git Commands

- Get all not existing objects from remote repository

```
git fetch https://github.com/user/repo
```

- Get and merge changed objects from remote repository

```
git pull
```

- Push the changes to a remote repository

```
git push
```

Workflow Patterns

Brief Overview

Git Workflows

- Workflows govern how the changes are committed and deployed
- Git does not need them, but we
- Popular workflows are **trunk-based development**, **feature branching** and **gitflow**
- Workflow selection depends on
  - What we are building
  - How we deploy it

...

Trunk Based Development

- Single branch - integrate into master/main
- Either directly or via short-lived branches
- Code can be deployed, without being released
- It is hidden behind feature flags
- Pros
  - Simple workflow; small commits; fast feedback; limited merge conflicts
- Cons
  - Need for feature flags; small teams; demands maturity (team & pipeline)

Feature Branching

- Long living main branch
- Short lived feature branches
- Code in main is always production ready
- Pros
  - Simple workflow; concurrent feature development
- Cons
  - small teams; robust pipeline; potentially difficult merge conflicts

...

GitFlow

- Multiple long living branches – main and develop
- Multiple short living branches – feature, release, hotfix
- Suitable for software with multiple supported versions
- Pros
  - Large teams; improved traceability; multiple versions are supported
- Cons
  - Difficult and complex; coordination and release management

...

GitFlow Mutations

- GitHub Flow
  - Two types of branches – main and features
  - Feature branches are merged directly to main
  - Suitable for web apps; APIs; SaaS; one version in prod
- GitLab Flow
  - Supports multiple environments
  - Multiple long living branches – main, test, prod, etc.
  - Short living feature branches

…

Practice: Version Control

Everything is Under Control ☺

Build Tools

Continuous Integration and More

The Need

- **Speed**
  - Build frequently and faster
- **Automation**
  - Build and deploy in an automated fashion
- **Predictable results**
  - Test each step and deploy only if everything is okay
- **Faster time-to-market**
  - Possibility to deliver to production at any time

The Landscape

- Popular Names
  - GitHub (Actions), GitLabCI, TravisCI, Jenkins, Gitea (Actions), TeamcityCI, Drone CI, etc.
- Offerings
  - **Hosted** solutions with various tiers (incl. free)
  - **On-premises** solutions with various tiers (incl. free)
  - **Mixed** – hosted and on-prem

Gitea

The Local GitHub

Gitea as a CI/CD Tool

- Evolved from a clone to a complete and capable solution
- Integrated CI/CD (Gitea Actions)
- Container-Native Execution
- Resource Efficient

...

GitHub Actions vs Gitea Actions

GitHub Actions

- YAML
- .github/workflows/*.yml
- Actions via marketplace
- Hosted
- Managed & self-hosted runners
- Built-in secrets
- Automatic scaling

Gitea Actions

- YAML (high compatibility)
- .gitea/workflows/*.yml
- Actions via URLs
- Self-hosted
- Self-hosted runners
- Built-in secrets
- Manual scaling

Jenkins

A Well-known Hero

What is Jenkins?

- An open-source automation server
- A platform for the Software Development Life Cycle (SDLC)
- Typically used to implement CI/CD
- Easy to use and highly adaptable
- Extensible and customizable
- Works on most common operating systems
- Considered lightweight

Key Definitions

- **Job**
  - Configured task in Jenkins. It is an old term
- **Project**
  - A task configured in Jenkins. It is the current term
- **Pipeline**
  - Special type of job created by a Pipeline plugin

Pipelines

- Old name is workflows
- Used for long running activity orchestration
- Can span on multiple nodes (slaves)
- It is a suite of plugins
- “Pipeline as code” is defined with a **Jenkinsfile**

Jenkinsfile

- It is used to define a pipeline
- Stored together with our code
- Two styles are supported – declarative and scripted
- It is written in Groovy

Declarative Jenkinsfile

Software University

It is required.

Specify which agent to run the pipeline or a stage. Can specify multiple nodes.

Mark the stages

Each stage describes a step in the SDLC

Steps are the actual work or actions to take place.
They are similar to “Build Step” in the Project Configuration view

Additional directives include **post**, **environment**, **triggers**, and **parameters**

```
pipeline {
  agents any
  stages {
    stage('Build') {
      steps {
        echo 'Step: Building...'
      }
    }
    stage('Test') {
      steps {
        echo 'Step: Testing...'
      }
    }
    stage('Deploy') {
      steps {
        echo 'Step: Deploying...'
      }
    }
  }
}
```

SCM Integration

- We store the pipeline code as **Jenkinsfile** in a repository
- During the build, Jenkins automatically checks out the file
- During job creation we point out where the **Jenkinsfile** is
- Offered trough plugins (GitHub one installed by default)
- Usually, requires a local client

Trigger Builds

- **Manually**
- **Scheduled**
  - With plugin or native
  - Three options
    - **Execute once** at a specific point in time
    - **Regular execution**
    - ***Regular checks against SCMs****, and execution if there is a change*
  - Execution interval is set with a **cron like syntaxis**
- **Webhooks**

Webhooks

- Webhooks link the build process with changes in the SCM
- There are **different plugins** that **expose endpoints** in Jenkins
- Some of them are specialized and others are more generic
- They are called from the SCM when there is a **push**, for example
- Some allow greater control like filtering, for example

Practice: CI Pipelines

Let’s Create a Few

Artifacts Management

Tools Landscape

Why and What

- **Immutable Versioning**
  - Ensures that what we tested is exactly what we deploy
- **Dependency Reliability**
  - Caching external libraries locally to protect our builds from external outages or deleted public packages
- **Faster Build Cycles**
  - Save time by storing intermediate build results
- **Security & Compliance**
  - Single point of truth for vulnerability scanning
- **Efficient Distribution**
  - Decouples stages - one build can be promoted through multiple environments

Gitea Packages

- **Multi-Protocol Support**
  - Native hosting for multiple package managers, including Docker/OCI, npm, PyPI, NuGet, Maven, and Cargo
- **Unified and Simplified Access Control**
  - Use the same organization and team permissions for our packages as for the repositories
- **Generic Storage**
  - Supports **generic packages** for storing .zip, .exe, or .iso files via a simple REST API
- **Storage Flexibility**
  - Scale from local disk storage to S3-compatible object storage
- **Cleanup & Retention**
  - Automated rules to prune old versions and snapshots

...

Docker Registry

- **Unified Storage (OCI Artifacts)**
  - Use the same registry to store container images, Helm charts, and LLM models
- **Simplified Tooling**
  - No need to learn new package manager CLIs
- **Storage Deduplication**
  - Designed to handle layers, leading to saving massive amounts of disk space
- **Immutable Versioning**
  - Allows using Image Digests (SHA-256 hashes) instead of just tags
- **Portable & Stateless**
  - Can be deployed as a single container and can be backed by simple local storage or robust cloud buckets (S3/GCS)

...

Practice: Artifacts

Build and Ship Them

Questions?

SoftUni

Software University | SoftUni AI | SoftUni Digital | SoftUni Creative | SoftUni Foundation | СофтУни БУДИТЕЛ | Finance Academy

Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg, about.softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg
- © Software University – https://softuni.bg